

# System Design Interview - راهنمای کامل

## برای افراد غیر کامپیوتر

این مجموعه شامل طراحی کامل ۵ سیستم کلیدی به زبان ساده و قابل فهم برای افرادی است که رشته‌شان کامپیوتر نیست. هر سیستم با تشبیه‌های دنیای واقعی، نمودارهای متنی و نکات مصاحبه‌ای ارائه شده است.

### هدف این مجموعه

- درک مفاهیم بدون نیاز به کدنویسی
- توانایی طراحی روی تخته سفید در مصاحبه
- آشنایی با اصطلاحات کلیدی سیستم دیزاین
- آمادگی برای سوالات سطح بالا (High-Level Design)

## سیستم ۱: طراحی سرویس کوتاه‌کننده لینک (URL Shortener)

### (Shortener)

### مشکل چیست؟

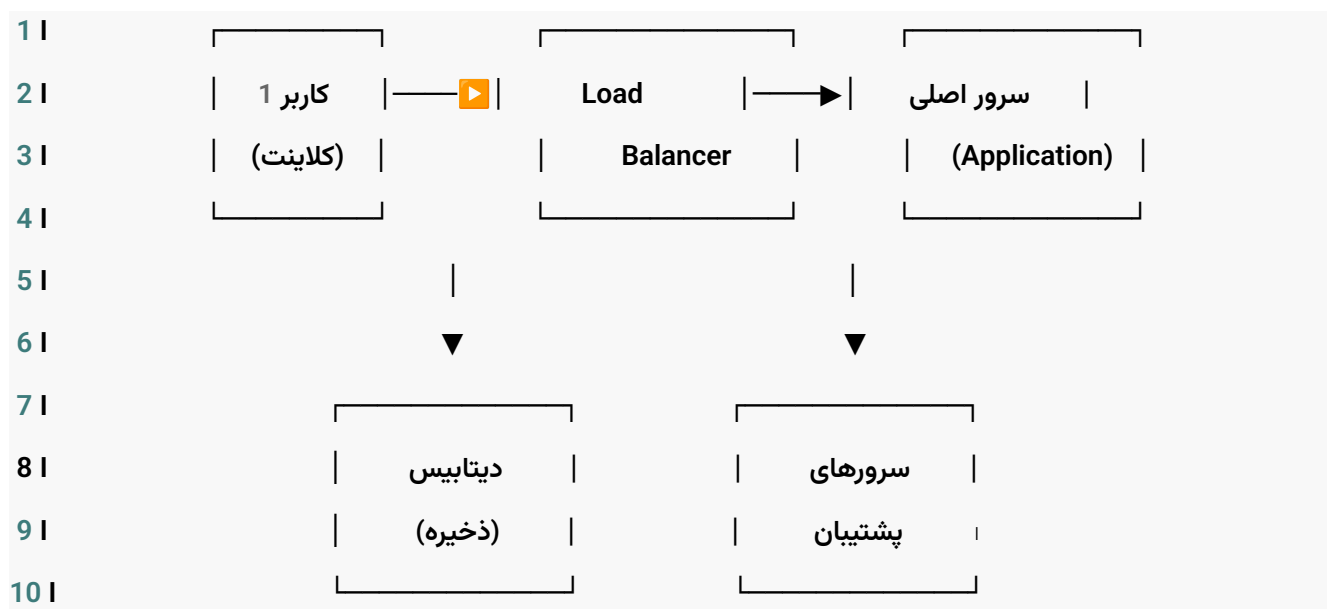
ما می‌خواهیم یک سرویس بسازیم که یک لینک بلند (مثل <https://www.example.com/blog/post/12345>) را به یک لینک کوتاه (مثل [tinyurl.com/abc123](https://tinyurl.com/abc123)) تبدیل کند. وقتی کاربر روی لینک کوتاه کلیک کند، به لینک اصلی هدایت شود.

## معماری کلی (با تشبیه دنیای واقعی) 🏗️

تصور کنید یک کتابخانه دارید:

- کتابدار (سرور): درخواست‌ها را دریافت می‌کند
- فهرست کتاب‌ها (دیتابیس): لینک بلند و کوتاه را ذخیره می‌کند
- کدهای مخفف (ID Generator): شماره‌های منحصر به فرد برای هر لینک تولید می‌کند

## نمودار سطح بالا (High-Level Diagram) 📊



تیکه ۶: Application Server و Load Balancer

## اجزای اصلی (Components)

### ۱. Load Balancer (منشی ورودی)

- وظیفه: توزیع درخواست‌ها بین چند سرور
- تشبیه: منشی یک شرکت که مشتریان را به کارشناسان مختلف ارجاع می‌دهد
- نکته مصاحبه: از Round Robin یا Least Connections استفاده می‌کنیم

### ۲. Application Server (سرور برنامه)

- وظیفه: پردازش منطق اصلی
- کارهایی که انجام می‌دهد:
- دریافت لینک بلند از کاربر
- تولید یک کد یکتا (مثل abc123)
- ذخیره کردن در دیتابیس
- برگرداندن لینک کوتاه به کاربر

### ۳. Database (دیتابیس)

- وظیفه: ذخیره‌سازی زوج‌های (لینک کوتاه ← لینک بلند)
- ساختار جدول:

|   |            |                         |            |
|---|------------|-------------------------|------------|
| 1 |            |                         |            |
| 2 | short_code | original_url            | created_at |
| 3 |            |                         |            |
| 4 | abc123     | https://example.com/... | 2024-01-01 |
| 5 | xyz789     | https://google.com/...  | 2024-01-02 |
| 6 |            |                         |            |

تیکه ۸: الگوریتم تولید کد کوتاه

## الگوریتم تولید کد کوتاه 🧠

### روش ۱: Base62 Encoding (توصیه شده)

• ایده: عددی را به حروف و اعداد تبدیل کنیم

• مثال: عدد 123456 را به w7e تبدیل می‌کنیم

• مزیت: خیلی کوتاه و خوانا

• تشبیه: مثل تبدیل شماره به کد پستی

### روش ۲: Hash Function (تابع هش)

• ایده: از لینک اصلی یک هش می‌گیریم

• مشکل: احتمال برخورد (Collision) وجود دارد

• تشبیه: مثل اثر انگشت که ممکن است دو نفر اثر انگشت یکسان داشته باشند (بسیار نادر)

## مقیاس پذیری (Scalability)

مشکل: اگر روزانه ۱۰۰ میلیون لینک بسازیم، دیتابیس پر می‌شود!

### راه حل ۱: Sharding (تقسیم دیتابیس)

• تشبیه: به جای یک انبار بزرگ، چندین انبار در شهرهای مختلف

• پیاده‌سازی: بر اساس اولین حرف کد کوتاه

1 کدهای شروع با A: دیتابیس

2 کدهای شروع با B: دیتابیس

تیکه ۱۰: Cache و سوالات مصاحبه ۱

### راه حل ۲: Cache (حافظه موقت)

• تشبیه: دفترچه یادداشت کنار دست برای لینک‌های پرتعداد

• ابزار: Redis یا Memcached

• نکته: ابتدا کش را چک می‌کنیم، اگر نبود به دیتابیس می‌رویم

## ! سوالات مصاحبه‌ای

**سوال ۱: چطور از تکراری شدن کدها جلوگیری می‌کنیم؟**

جواب: از یک شمارنده مرکزی (مثل Redis INCR) استفاده می‌کنیم تا هر بار یک عدد یکتا بگیریم و آن را به Base62 تبدیل کنیم.

## سوال ۲: اگر کاربر لینک تکراری بدهد چه می‌شود؟

جواب: دو رویکرد:

• رویکرد ۱: همیشه لینک جدید بساز (ساده‌تر)

• رویکرد ۲: چک کن اگر لینک قبلاً ثبت شده، همان کد را برگردان (پیچیده‌تر)

## سوال ۳: آمار بازدید (Analytics) را چگونه ثبت کنیم؟

جواب: یک جدول جداگانه برای ثبت هر کلیک:

|   |            |             |            |
|---|------------|-------------|------------|
| 1 |            |             |            |
| 2 | short_code | user_ip     | click_time |
| 3 |            |             |            |
| 4 | abc123     | 192.168.1.1 | 2024-01-01 |
| 5 |            |             |            |

تیکه ۱۲: خلاصه سیستم ۱

## خلاصه یک صفحه‌ای برای مصاحبه

سوال: یک سرویس کوتاه‌کننده لینک طراحی کنید.

پاسخ سطح بالا:

• Load Balancer برای توزیع ترافیک

• سرورهای Application برای پردازش

• دیتابیس برای ذخیره (با Sharding و Cache)

• ID Generator برای تولید کدهای یکتا

• سیستم کش برای لینک‌های پرطرفدار

تخمین اعداد:

• ۱۰۰ میلیون لینک جدید در روز

• هر لینک متوسط ۱KB فضا

• ۱۰۰GB فضای ذخیره‌سازی در روز!

## سیستم ۲: طراحی سیستم پیام‌رسان (مثل تلگرام/واتساپ)

مشکل چیست؟ 

می‌خواهیم سیستمی طراحی کنیم که کاربران بتوانند پیام‌های متنی، تصاویر و فایل‌ها را به صورت لحظه‌ای با هم به اشتراک بگذارند. ویژگی‌های کلیدی: ارسال پیام، دریافت لحظه‌ای، وضعیت آنلاین/آفلاین، تاریخچه پیام‌ها.

تشبیه دنیای واقعی 

سیستم پیام‌رسان مثل یک اداره پست با ویژگی‌های ویژه است:

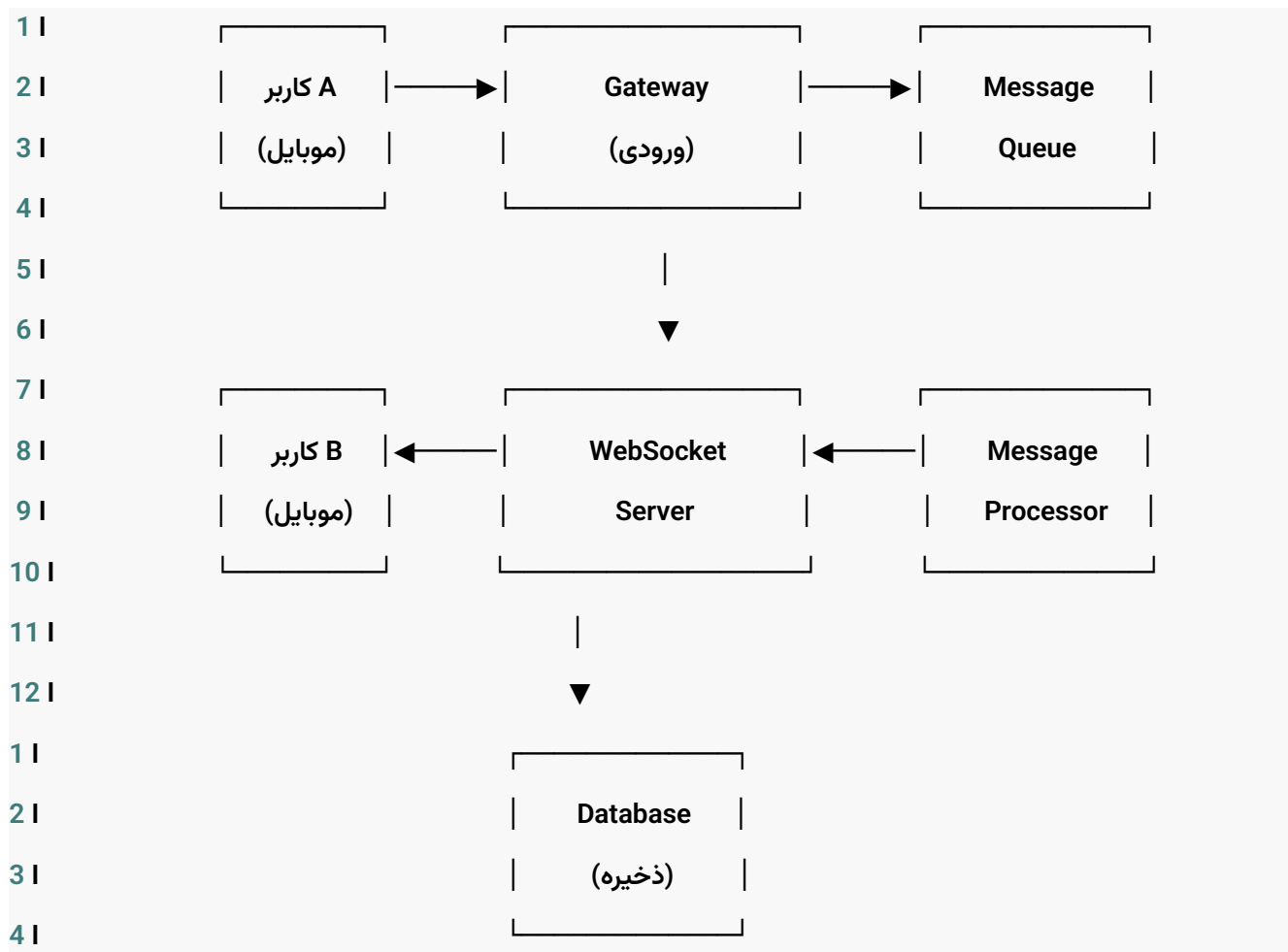
• کاربر: فرستنده/گیرنده نامه

• صندوق پستی (سرور): محل ذخیره نامه‌ها

• پستچی (WebSocket): نامه را فوری می‌رساند

• آدرس‌نامه (دیتابیس): لیست مخاطبین

## معماری کلی



تیکه ۱۵: Message Queue و Gateway

## اجزای اصلی

### ۱. Gateway (ورودی سیستم)

•وظیفه: اولین نقطه برخورد با کاربر

• تشبیه: ورودی یک ساختمان بزرگ

• کارها:

• احراز هویت (Authentication)

• هدایت درخواست‌ها به سرور مناسب

• محدودیت نرخ درخواست (Rate Limiting)

## ۲. Message Queue (صف پیام‌ها)

• وظیفه: پیام‌ها را در صف قرار دهد تا یکی‌یکی پردازش شوند

• تشبیه: صف نانوايي - همه وارد می‌شوند ولی یکی‌یکی خارج می‌شوند

• چرا مهم است؟ اگر یک کاربر ۱۰۰۰ پیام بفرستد، سرور را خراب نمی‌کند

## ۳. WebSocket Server (سرور اتصال لحظه‌ای)

• وظیفه: ارتباط دوطرفه و لحظه‌ای با کاربر

• تشبیه: تلفن همراه - هر دو طرف می‌توانند همزمان صحبت کنند

• تفاوت با HTTP: HTTP مثل پست است (بفرست و منتظر جواب باش)، WebSocket مثل تلفن (همیشه وصل هستی)

## ۴. Message Processor (پردازشگر پیام)

• وظیفه: گرفتن پیام از صف و ذخیره کردن در دیتابیس

• تشبیه: کارمند پشت باجه که نامه‌ها را بایگانی می‌کند

## ۵. Database (دیتابیس)

• ساختار پیام:

|   |    |           |             |          |             |  |
|---|----|-----------|-------------|----------|-------------|--|
| 1 |    |           |             |          |             |  |
| 2 | id | sender_id | receiver_id | content  | timestamp   |  |
| 3 |    |           |             |          |             |  |
| 4 | 1  | 101       | 102         | 12:30:00 | سلام چطوری؟ |  |
| 5 | 2  | 102       | 101         | 12:30:15 | خوبم تو؟    |  |
| 6 |    |           |             |          |             |  |

## مدیریت وضعیت آنلاین/آفلاین

### روش: Heartbeat Mechanism (ضربان قلب)

• تشبیه: مثل چراغ حیات خانه - اگر روشن باشد، شما خانه‌اید

• کار: هر ۵ ثانیه، کاربر یک سیگنال ("من زنده‌ام") به سرور می‌فرستد

• اگر ۱۵ ثانیه سیگنال نیاید → کاربر آفلاین محسوب می‌شود

تیکه ۱۸: مقیاس‌پذیری و امنیت سیستم ۲

## ذخیره‌سازی پیام‌ها (مقیاس‌پذیری)

مشکل: هر روز میلیاردها پیام تولید می‌شود!

## راه حل ۱: شاردينگ بر اساس User ID

دیتابیس ۱: کاربران ۱ تا ۱۰۰۰

دیتابیس ۲: کاربران ۱۰۰۱ تا ۲۰۰۰

## راه حل ۲: جداسازی پیام‌های قدیمی و جدید

• پیام‌های ۳ ماه اخیر: در دیتابیس اصلی (سریع)

• پیام‌های قدیمی‌تر: در دیتابیس بایگانی (آرشیو)

## امنیت

### ۱. احراز هویت (Authentication)

• هر کاربر یک Token دارد (مثل کارت شناسایی)

• هر درخواست باید Token معتبر داشته باشد

### ۲. رمزنگاری (Encryption)

• توصیه: پیام‌ها را در مسیر (HTTPS) و در دیتابیس رمز کنید

• تشبیه: نامه را در پاکت قفل‌دار بگذارید

تیکه ۱۹: سوالات مصاحبه سیستم ۲ (بخش اول)

## ! سوالات مصاحبه‌ای

**سوال ۱: چطور پیام را به چندین نفر (گروهی) بفرستیم؟**

جواب: به جای ارسال به هر کاربر، پیام را یک بار در دیتابیس ذخیره می‌کنیم و یک جدول `group_members` داریم که مشخص می‌کند هر کاربر در کدام گروه‌هاست. فقط کافی است پیام را به همه اعضای گروه اطلاع‌رسانی کنیم.

**سوال ۲: اگر کاربر آفلاین باشد، پیام را چطور دریافت می‌کند؟**

جواب:

- پیام در دیتابیس ذخیره می‌شود با وضعیت `delivered: false`
- وقتی کاربر آنلاین شد، سرور پیام‌های تحویل نشده را برایش می‌فرستد
- کاربر پیام را دریافت می‌کند و وضعیت به `delivered: true` تغییر می‌کند

**سوال ۳: چطور از دوبار ارسال شدن پیام جلوگیری کنیم؟ (Idempotency)**

جواب: هر پیام یک `message_id` یکتا دارد. سرور چک می‌کند اگر پیامی با این ID قبلاً پردازش شده، آن را نادیده می‌گیرد.

**سوال ۴: تفاوت WebSocket و HTTP در این سیستم چیست؟**

جواب:

- HTTP: برای ارسال پیام (فرستنده → سرور)
- WebSocket: برای دریافت لحظه‌ای پیام (سرور → گیرنده)

## خلاصه یک صفحه‌ای

سوال: یک سیستم پیام‌رسان طراحی کنید.

پاسخ سطح بالا:

- Gateway برای ورود و احراز هویت
- Message Queue برای مدیریت بار
- WebSocket Server برای ارتباط لحظه‌ای
- Message Processor برای ذخیره‌سازی
- دیتابیس با Sharding و Cache

تخمین اعداد:

- ۵۰۰ میلیون کاربر فعال
- هر کاربر روزانه ۵۰ پیام → ۲۵ میلیارد پیام در روز
- نیاز به سیستم ذخیره‌سازی ابری عظیم

## سیستم ۳: طراحی شبکه اجتماعی (مثل توییتر/X)

### مشکل چیست؟

طراحی یک شبکه اجتماعی که کاربران بتوانند:

- پست (Tweet) منتشر کنند
- پست دیگران را بخوانند (Timeline/Feed)
- دنبال کنندگان (Followers) داشته باشند

• پست‌ها را لایک و کامنت کنند

## تشبیه دنیای واقعی 🏠

شبکه اجتماعی مثل یک شهر بزرگ با روزنامه‌های شخصی است:

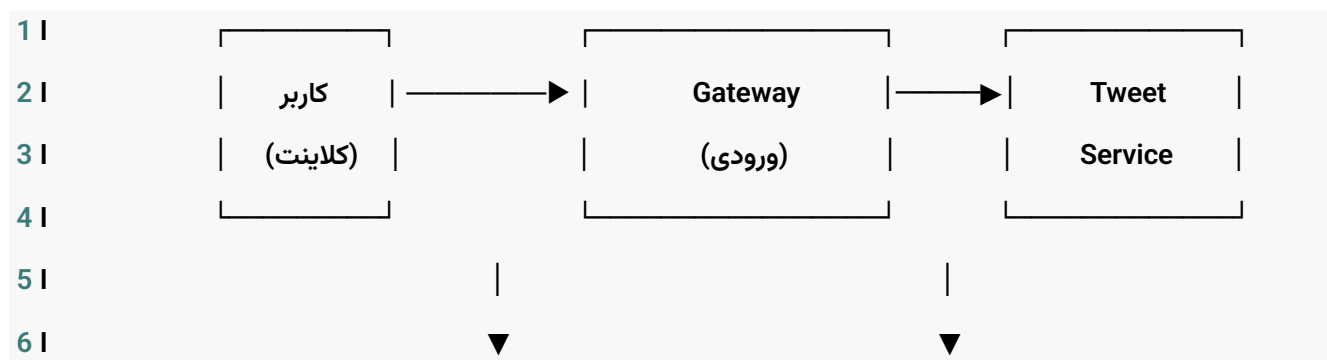
• هر کاربر: یک روزنامه‌نگار

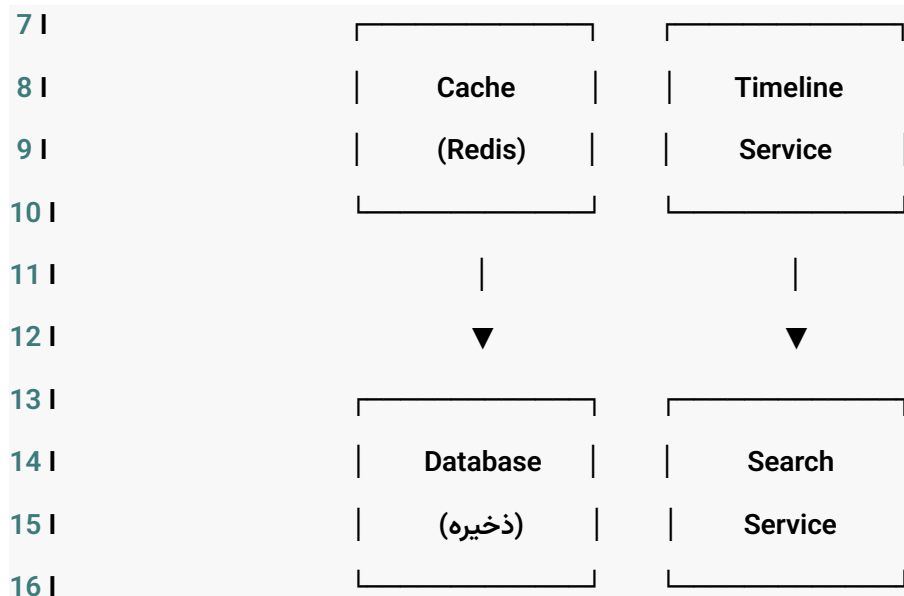
• تویییت: یک مقاله کوتاه

• دنبال کردن: مشترک شدن در روزنامه آن شخص

• تایم‌لاین: صفحه اول روزنامه شخصی شما

## معماری کلی 📊





تیکه ۲۳: Timeline Service و Tweet

## اجزای اصلی

### ۱. Tweet Service (سرویس پست)

•وظیفه: ایجاد، حذف و دریافت توییت‌ها

•تشبیه: روزنامه‌نگار که مقاله می‌نویسد

•کارها:

•ذخیره توییت در دیتابیس

•ارسال توییت به سیستم تایم‌لاین

### ۲. Timeline Service (سرویس تایم‌لاین)

•وظیفه: نمایش آخرین پست‌های افرادی که دنبال می‌کنید

•تشبیه: صفحه اول روزنامه شما

### ۳. Follow Service (سرویس دنبال کردن)

•وظیفه: مدیریت روابط بین کاربران (چه کسی چه کسی را دنبال می‌کند)

•ساختار:

|   |             |              |            |
|---|-------------|--------------|------------|
| 1 |             |              |            |
| 2 | follower_id | following_id | created_at |
| 3 |             |              |            |
| 4 | 101         | 102          | 2024-01-01 |
| 5 | 101         | 103          | 2024-01-02 |
| 6 |             |              |            |

## 🧠 دو رویکرد طراحی تایم‌لاین

### رویکرد ۱: Pull-based (کشیدن - توصیه شده برای توییتر)

•تشبیه: شما هر روز به غرفه‌های مختلف می‌روید و روزنامه می‌خرید

•کار: وقتی تایم‌لاین را باز می‌کنید، سرور توییت‌های افرادی که دنبال می‌کنید را جمع‌آوری می‌کند

•مزیت: ساده و کم‌هزینه

•عیب: اگر ۵۰۰۰ نفر را دنبال کنید، کند می‌شود

### رویکرد ۲: Push-based (فشاردن - توصیه شده برای اینستاگرام)

•تشبیه: روزنامه‌ها برای شما پست می‌شوند (اشتراک)

• کار: وقتی کسی توییت می‌زند، برای همه دنبال‌کنندگان یک کپی در تایم‌لاینشان ذخیره می‌شود

• مزیت: بسیار سریع

• عیب: اگر سلبریتی با ۱۰۰ میلیون دنبال‌کننده توییت بزند، ۱۰۰ میلیون کپی ذخیره می‌شود!

تیکه ۲۵: ترکیب رویکردها و Caching

ترکیب هر دو (توصیه نهایی)

• برای سلبریتی‌ها: Pull-based (چون تعداد کپی‌ها زیاد است)

• برای کاربران عادی: Push-based (چون دنبال‌کننده‌ها کم است)

## کش (Caching) در شبکه اجتماعی

چرا کش مهم است؟

• توییت‌های داغ (Viral) میلیون‌ها بار دیده می‌شوند

• بدون کش، دیتابیس می‌سوزد!

لایه‌های کش

1 **مرورگر** (Browser Cache) لایه ۱: کش

2 |

3 **۲** (برای عکس‌ها و فیلم‌ها) CDN: لایه

4 |

5 **۳** (برای توییت‌های داغ) Redis: لایه

6 |

7 لایه ۴: دیتابیس اصلی

تیکه ۲۶: استراتژی کش و جستجو

## استراتژی کش: Cache-Aside

- کاربر توپیت را درخواست می‌کند
- اول کش را چک می‌کنیم
- اگر بود → برمی‌گردانیم (خیلی سریع)
- اگر نبود → از دیتابیس می‌خوانیم و در کش ذخیره می‌کنیم

## جستجو (Search)

- مشکل: جستجوی توپیت بین میلیاردها توپیت
- راه حل: Elasticsearch
- تشبیه: یک کتابخانه با فهرست‌های پیشرفته
- چرا؟ برای جستجوی متن و هشتک‌ها عالی است
- کار: توپیت‌ها را Index (فهرست‌بندی) می‌کند تا جستجو سریع شود

## ! سوالات مصاحبه‌ای

سوال ۱: چطور توپیت‌های داغ (Trending) را پیدا می‌کنیم؟

جواب:

- هر توپیت یک وزن دارد (بر اساس لایک، کامنت، بازتوپییت)
- سیستم یک کرون جاب (هر ۵ دقیقه) محاسبه می‌کند

• از الگوریتم‌های کشف روند مثل Count-Min Sketch استفاده می‌کنیم

**سوال ۲: چطور از اسپم و محتوای نامناسب جلوگیری کنیم؟**

جواب:

• فیلتر کلمات کلیدی

• سیستم یادگیری ماشین برای تشخیص محتوای نامناسب

• محدودیت نرخ (Rate Limiting) - هر کاربر در روز فقط ۱۰۰۰ توییت

**سوال ۳: اگر یک سلبریتی ۲۰۰ میلیون دنبال‌کننده داشته باشد، چطور توییت می‌فرستد؟**

جواب:

• از رویکرد Pull استفاده می‌کنیم

• توییت را فقط یک بار ذخیره می‌کنیم

• وقتی هر کاربر تایم‌لاینش را باز می‌کند، توییت را می‌خواند

## خلاصه یک صفحه‌ای

سوال: یک شبکه اجتماعی مثل توییتر طراحی کنید.

پاسخ سطح بالا:

• Gateway برای ورود و احراز هویت

• Tweet Service برای مدیریت پست‌ها

• Timeline Service با ترکیب Push/Pull

• Cache (Redis) برای سرعت

•Elasticsearch برای جستجو

•دیتابیس با Sharding بر اساس User ID

تخمین اعداد:

•۵۰۰ میلیون کاربر فعال

•هر کاربر روزانه ۱۰ توییت → ۵ میلیارد توییت در روز

•هر توییت متوسط ۱۴۰ کاراکتر → ۷۰۰GB متن در روز!

## سیستم ۴: طراحی سیستم فروشگاه‌های (مثل دیجیکالا)

### مشکل چیست؟

طراحی یک فروشگاه اینترنتی که کاربران بتوانند:

•محصولات را مرور کنند

•به سبد خرید اضافه کنند

•سفارش ثبت کنند

•پرداخت انجام دهند

•وضعیت سفارش را پیگیری کنند

### تشبیه دنیای واقعی

فروشگاه اینترنتی مثل یک فروشگاه بزرگ فیزیکی است:

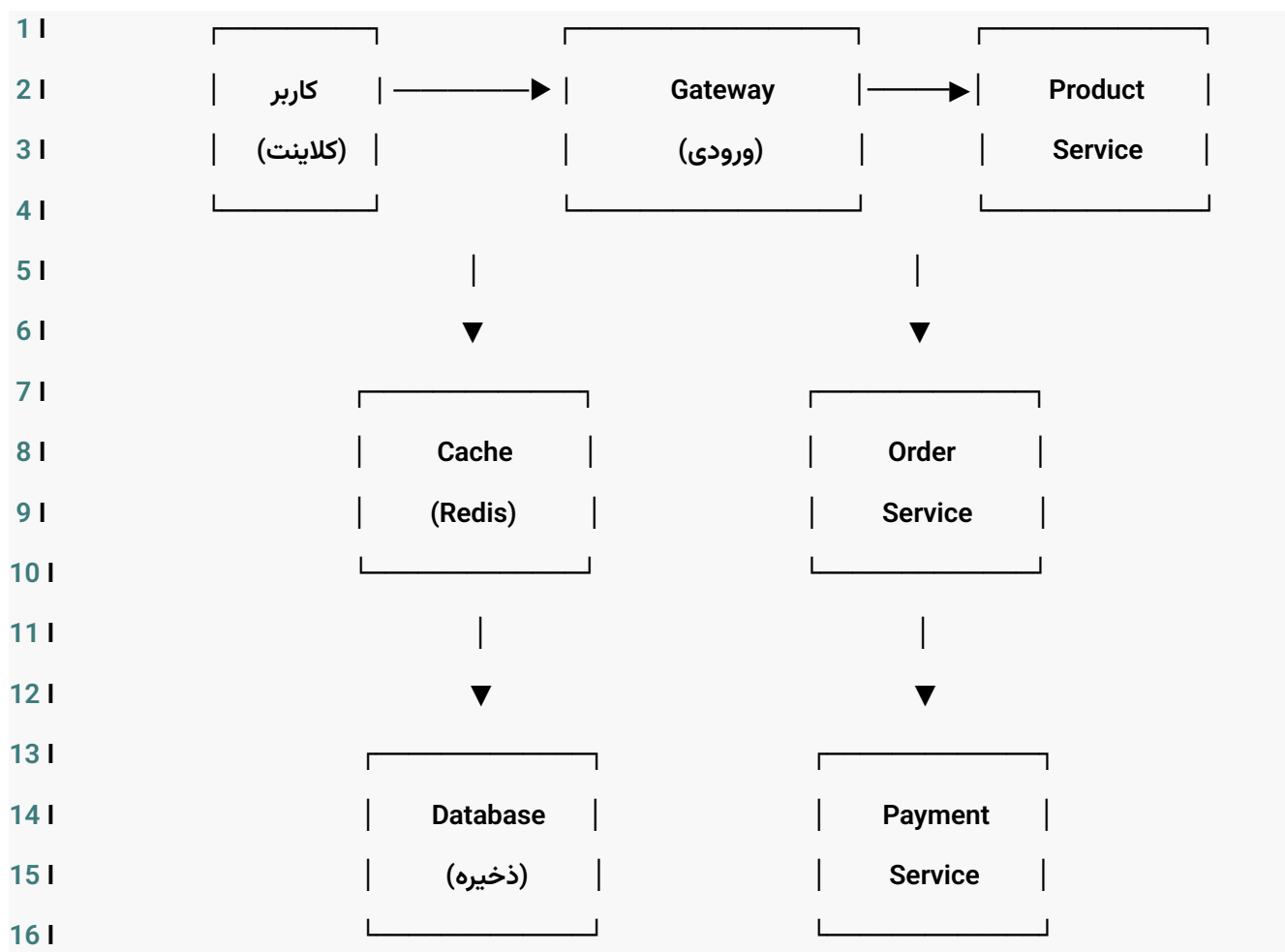
• محصولات: کالاهای موجود در قفسه‌ها

• سبد خرید: سبد خرید فیزیکی شما

• انبار: انبار پستی فروشگاه

• پرداخت: صندوق فروش

## معماری کلی



تیکه ۳۰: Product و Cart Service

## اجزای اصلی

### ۱. Product Service (سرویس محصولات)

•وظیفه: مدیریت اطلاعات محصولات

•تشبیه: قفسه‌های فروشگاه با برچسب قیمت

•اطلاعات محصول:

|   |            |         |            |              |  |
|---|------------|---------|------------|--------------|--|
| 1 |            |         |            |              |  |
| 2 | product_id | name    | price      | stock        |  |
| 3 |            |         |            |              |  |
| 4 | 101        | 100 عدد | 20,000,000 | گوشی سامسونگ |  |
| 5 | 102        | 500 عدد | 2,500,000  | هدفون بلوتوث |  |
| 6 |            |         |            |              |  |

### ۲. Cart Service (سرویس سبد خرید)

•وظیفه: مدیریت سبد خرید کاربر

•تشبیه: سبد خرید در فروشگاه

•نکته: معمولاً در Redis ذخیره می‌شود (سریع و موقت)

تیکه ۳۱: Order و Payment Service و مدیریت موجودی

### ۳. Order Service (سرویس سفارشات)

•وظیفه: ثبت سفارش نهایی

• تشبیه: خرید نهایی در صندوق

• مراحل:

1. دریافت اطلاعات سبد خرید

2. چک کردن موجودی

3. محاسبه قیمت نهایی

4. ثبت سفارش در دیتابیس

## ۴. Payment Service (سرویس پرداخت)

• وظیفه: پردازش پرداخت

• تشبیه: کارت خوان فروشگاه

• نکته: معمولاً با درگاه بانکی ارتباط برقرار می‌کند

## مدیریت موجودی (Inventory Management)

مشکل: دو کاربر همزمان آخرین محصول را می‌خرند! (Race Condition)

### راه حل: قفل خوش‌بینانه (Optimistic Lock)

• هر محصول یک version دارد

• وقتی کاربر می‌خواهد بخرد، موجودی را چک می‌کند

• اگر موجودی  $> 0$  بود، خرید انجام می‌شود

• version را ۱ افزایش می‌دهیم

• اگر version تغییر کرده بود (کاربر دیگری خرید)، سفارش را Cancel می‌کنیم

## راه حل: قفل بدبینانه (Pessimistic Lock)

• وقتی کاربر محصولی را به سبد خرید اضافه می‌کند، محصول را قفل می‌کنیم

• کاربر دیگری نمی‌تواند آن را بخرد

• بعد از ۱۵ دقیقه، قفل آزاد می‌شود

## پرداخت و تراکنش (Transaction)

### ویژگی‌های ACID (مهم در مصاحبه)

| ویژگی       | توضیح                             | تشبیه                                                     |
|-------------|-----------------------------------|-----------------------------------------------------------|
| Atomicity   | یا همه مراحل انجام شود یا هیچکدام | اگر پول از حساب شما کم شد، باید به حساب فروشنده واریز شود |
| Consistency | داده‌ها همیشه معتبر هستند         | موجودی منفی نمی‌شود                                       |
| Isolation   | تراکنش‌ها از هم جدا هستند         | خرید شما بر خرید من تأثیر نمی‌گذارد                       |
| Durability  | بعد از انجام، از بین نمی‌رود      | حتی با قطع برق، سفارش ثبت شده باقی می‌ماند                |

### مدیریت تراکنش در کد

```

1 try {
2   start transaction
3   check inventory
4   decrease inventory
5   create order
6   process payment
7   commit transaction
8 } catch (Exception e) {

```

9 rollback transaction // همه چیز به حالت اول برمی‌گردد

10 }

تیکه ۳۳: توصیه‌ها و سوالات مصاحبه ۱

## توصیه‌ها و تخفیف‌ها (Recommendation)

سیستم Recommendation چطور کار می‌کند؟

• سابقه خرید کاربر: محصولات مشابه خریده است

• تخفیف‌های فصلی: مثل تخفیف نوروزی

• محصولات پرفروش: دیگران چه خریده‌اند

ذخیره‌سازی اطلاعات کاربر برای شخصی‌سازی

• تشبیه: فروشنده که مشتریان همیشگی را می‌شناسد

## ! سوالات مصاحبه‌ای

سوال ۱: چطور از فروش محصولی که موجودی ندارد جلوگیری کنیم؟

جواب: از Inventory Service مستقل استفاده می‌کنیم که:

• قبل از ثبت سفارش، موجودی را چک می‌کند

• موجودی را به‌روز نگه می‌دارد

• اگر موجودی کم بود، به کاربر هشدار می‌دهد

## سوال ۲: اگر پرداخت ناموفق بود، چه می‌شود؟

جواب:

- تمام تراکنش Rollback می‌شود
- موجودی به حالت اول برمی‌گردد
- سفارش با وضعیت failed ذخیره می‌شود
- به کاربر پیام خطا نمایش داده می‌شود

## سوال ۳: چطور از کلاهبرداری (Fraud) جلوگیری کنیم؟

جواب:

- محدودیت نرخ (Rate Limiting)
- تشخیص آدرس‌های IP مشکوک
- تأیید دو مرحله‌ای برای خریدهای بزرگ
- الگوریتم‌های تشخیص الگوی خرید غیرعادی

## سوال ۴: سبد خرید را در کجا ذخیره کنیم؟ دیتابیس یا کش؟

جواب:

- پاسخ صحیح: در Redis (کش)
- دلیل: خیلی سریع است و موقتی بودن سبد خرید را نشان می‌دهد
- نکته: اگر کاربر لاگین کرده باشد، سبد خرید در دیتابیس هم بکاپ می‌شود

## خلاصه یک صفحه‌ای

سوال: یک فروشگاه اینترنتی مثل دیجیکالا طراحی کنید.

پاسخ سطح بالا:

• Gateway برای ورود و احراز هویت

• Product Service با کش (Redis)

• Redis در Cart Service

• Order Service با تراکنش ACID

• Payment Service متصل به درگاه بانکی

• Inventory Service برای مدیریت موجودی

تخمین اعداد:

۱۰۰ میلیون محصول

۱۰۰۰ میلیون کاربر

۱۰ میلیون سفارش در روز

۵۰۰ میلیون درخواست محصول در روز

## سیستم ۵: طراحی پلتفرم اشتراک ویدیو (مثل )

## یوتیوب/آپارات)

## مشکل چیست؟

طراحی پلتفرمی که کاربران بتوانند:

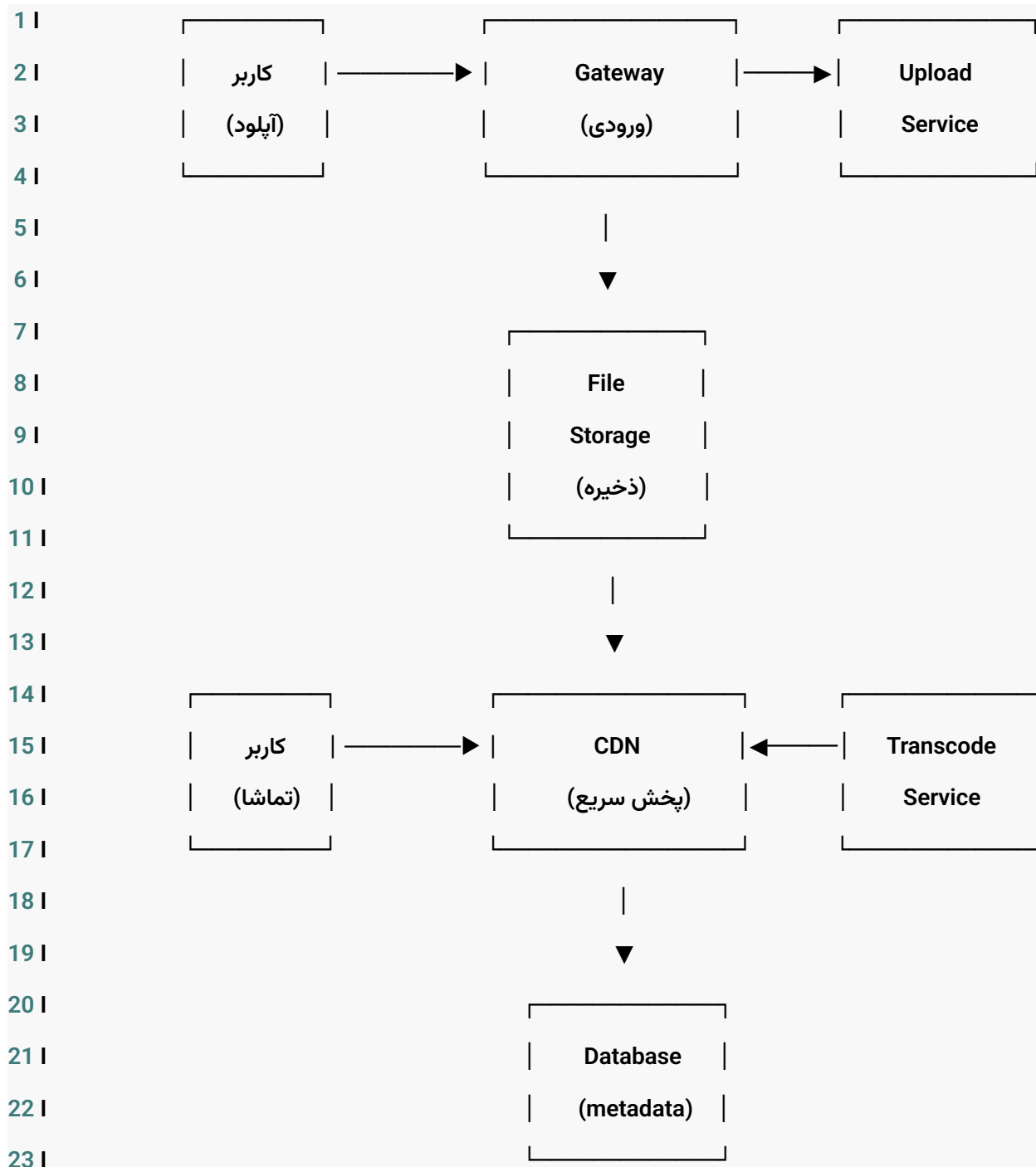
- ویدیو آپلود کنند
- ویدیو تماشا کنند (استریم)
- ویدیو را لایک/کامنت کنند
- کیفیت ویدیو را تغییر دهند (۴۸۰p, 720p, 1080p)

## تشبیه دنیای واقعی 🏠

پلتفرم ویدیو مثل یک شبکه تلویزیون با کتابخانه فیلم است:

- ویدیو: یک فیلم سینمایی
- آپلود: کارگردان فیلم را به تلویزیون می‌دهد
- تماشا: تماشاگر کانال را عوض می‌کند
- کیفیت‌های مختلف: SD, HD, 4K

## معماری کلی 📊



تیکه ۳۷: Upload, File Storage و Transcode Service

## اجزای اصلی

### ۱. Upload Service (سرویس آپلود)

- وظیفه: دریافت ویدیو از کاربر
- تشبیه: دریافت فیلم توسط تلویزیون
- چالش: ویدیوها بزرگ هستند (GBها)
- راه حل: آپلود تکه تکه (Chunked Upload)

### ۲. File Storage (ذخیره سازی فایل)

- وظیفه: ذخیره ویدیوها در فضای ابری
- ابزار: AWS S3، Google Cloud Storage
- تشبیه: بایگانی فیلم ها در انبار

### ۳. Transcode Service (سرویس تبدیل کیفیت)

- وظیفه: ویدیو را به کیفیت های مختلف تبدیل کند
- تشبیه: تبدیل فیلم به فرمت های مختلف
- کیفیت ها: ۲۴۰p, ۴۸۰p, ۷۲۰p, ۱۰۸۰p, ۴K

### ۴. CDN (شبکه توزیع محتوا)

- وظیفه: پخش سریع ویدیو به کاربران در سراسر جهان
- تشبیه: چندین سینما در شهرهای مختلف که یک فیلم را نمایش می دهند

•مزیت: ویدیو به نزدیک‌ترین سرور به کاربر می‌رود

## ۵. Metadata Service (سرویس اطلاعات)

•وظیفه: ذخیره اطلاعات ویدیو (عنوان، توضیحات، تعداد بازدید)

•تشبیه: جلد و برچسب پشت فیلم

## معماری آپلود ویدئو 🧠

### فرآیند آپلود تکه‌تکه (Chunked Upload)

- 1 | مرحله ۱: کاربر شروع به آپلود می‌کند
- 2
- 3 | تقسیم می‌شود MB مرحله ۲: ویدیو به تکه‌های ۲
- 4
- 5 | مرحله ۳: هر تکه جداگانه آپلود می‌شود
- 6
- 7 | مرحله ۴: سرور تکه‌ها را دریافت و ترکیب می‌کند
- 8
- 9 | مرحله ۵: اگر آپلود قطع شد، از همان تکه ادامه می‌دهد

## مزیت

•اگر اینترنت قطع شود، کل ویدیو از دست نمی‌رود

•می‌توان سرعت آپلود را بهینه کرد

تیکه ۳۹: معماری پخش ویدئو (Streaming)

## معماری پخش ویدئو (Streaming) 🎬

### پروتکل HLS (HTTP Live Streaming)

تشبیه: یک کتاب به چندین فصل تقسیم شده است



### مزیت

- کاربر نیازی به دانلود کل ویدئو ندارد
- ویدئو تکه‌تکه پخش می‌شود
- کیفیت بر اساس سرعت اینترنت تغییر می‌کند (Adaptive Bitrate)

تیکه ۴۰: ذخیره‌سازی و جستجوی ویدیو

## ذخیره‌سازی ویدیوها

آمار تقریبی:

- هر دقیقه ویدیو با کیفیت ۱۰۸۰p  $\approx$  ۱۰۰MB
- ۱۰۰۰ ویدیو در روز  $\approx$  ۱۰۰GB فضای ذخیره‌سازی
- بعد از ۱ سال  $\approx$  ۳۶TB فضای ذخیره‌سازی!

### راه‌حل‌های ذخیره‌سازی

#### ۱. ذخیره‌سازی سرد (Cold Storage)

- تشبیه: بایگانی فیلم‌های قدیمی در انبار
- ویژگی: ارزان ولی کند
- کاربرد: ویدیوهای قدیمی (بیش از ۱ سال)

#### ۲. ذخیره‌سازی گرم (Hot Storage)

- تشبیه: فیلم‌های جدید در قفسه اصلی
- ویژگی: گران ولی سریع
- کاربرد: ویدیوهای جدید و پرطرفدار

۳. سیستم توزیع شده (Distributed Storage)

• تشبیه: فیلم‌ها در چندین سینما پخش شده‌اند

• ابزار: HDFS یا GlusterFS

## جستجوی ویدیو

• چالش: جستجو بین میلیون‌ها ویدیو

• راه حل: Elasticsearch

• تشبیه: فهرست کتابخانه با سیستم جستجوی سریع

• اطلاعات Index شده:

• عنوان ویدیو

• توضیحات

• هشتک‌ها

• کامنت‌های محبوب

## سیستم پیشنهاد ویدیو (Recommendation)

چطور یوتیوب ویدیو پیشنهاد می‌دهد؟

• سابقه تماشای شما

• ویدیوهای مشابه (هم‌تگ)

• ویدیوهای محبوب (تعداد بازدید زیاد)

• ویدیوهای دوستان (اگر دنبال می‌کنید)

الگوریتم ساده:

$$1 \text{ score} = (\text{views} \times 0.3) + (\text{likes} \times 0.2) + (\text{comments} \times 0.1) + (\text{watch\_time} \times 0.4)$$

## ! سوالات مصاحبه‌ای

سوال ۱: چطور ویدیو را با سرعت کم اینترنت پخش کنیم؟

جواب:

- از Adaptive Bitrate Streaming استفاده می‌کنیم
- سرعت اینترنت کاربر را اندازه می‌گیریم
- کیفیت مناسب را انتخاب می‌کنیم (اگر سرعت کم است  $\rightarrow 480p$ )

سوال ۲: چطور از کپی‌رایت جلوگیری کنیم؟

جواب:

- سیستم Content ID (مثل یوتیوب)
- مقایسه هش ویدیوها با ویدیوهای ثبت شده
- تشخیص خودکار صدا و تصویر با AI

سوال ۳: آپلود همزمان میلیون‌ها ویدیو در روز را چطور مدیریت کنیم؟

جواب:

- Message Queue (مثل RabbitMQ) برای صف‌بندی
- چندین Transcode Worker برای پردازش موازی
- Auto-scaling در فضای ابری

## خلاصه یک صفحه‌ای

سوال: یک پلتفرم اشتراک ویدیو مثل یوتیوب طراحی کنید.

پاسخ سطح بالا:

- Gateway برای ورود و احراز هویت
- Upload Service با آپلود تکه‌تکه
- File Storage (S3/Google Cloud)
- Transcode Service برای کیفیت‌های مختلف
- CDN برای پخش سریع
- Metadata Service برای اطلاعات ویدیو
- Recommendation System برای پیشنهاد ویدیو

تخمین اعداد:

- ۲۰ میلیارد کاربر در ماه
- ۵۰۰ ساعت ویدیو در هر دقیقه آپلود می‌شود
- ۱۰ میلیارد ساعت ویدیو در روز تماشا می‌شود
- نیاز به فضای ذخیره‌سازی در مقیاس Exabyte

## جمع‌بندی نهایی برای مصاحبه

### الگوی پاسخ به هر سوال System Design

۱. درک نیازمندی‌ها (Requirements)

• فانکشنال (Functional) (چه کاری باید انجام دهد)

• غیر فانکشنال (Non-Functional) (مقیاس، امنیت، سرعت)

2. طراحی سطح بالا (High-Level Design)

• رسم نمودار با مستطیل و فلش

• توضیح هر کامپوننت

3. جزئیات مهم (Core Components)

• دیتابیس (نوع، شاردینگ)

• کش (Redis)

• پیام‌رسانی (Message Queue)

4. مقیاس‌پذیری (Scalability)

• چطور با افزایش کاربران کار می‌کند؟

• چطور سرعت را حفظ می‌کند؟

5. تخمین اعداد (Back-of-the-envelope)

• تعداد کاربران

• فضای ذخیره‌سازی

• پهنای باند

## منابع تکمیلی برای شما

• ویدیوهای کانال ByteByteGo (انیمیشن‌های فوق‌العاده)

• Grokking the System Design Interview (کتاب)

• کانال Gaurav Sen (یوتیوب - با زیرنویس فارسی)

• System Design Interview - Alex Xu (کتاب دو جلدی)

## سیستم ۶: طراحی سیستم بانکی (مثل بانک ملی/درگاه پرداخت)

### مشکل چیست؟

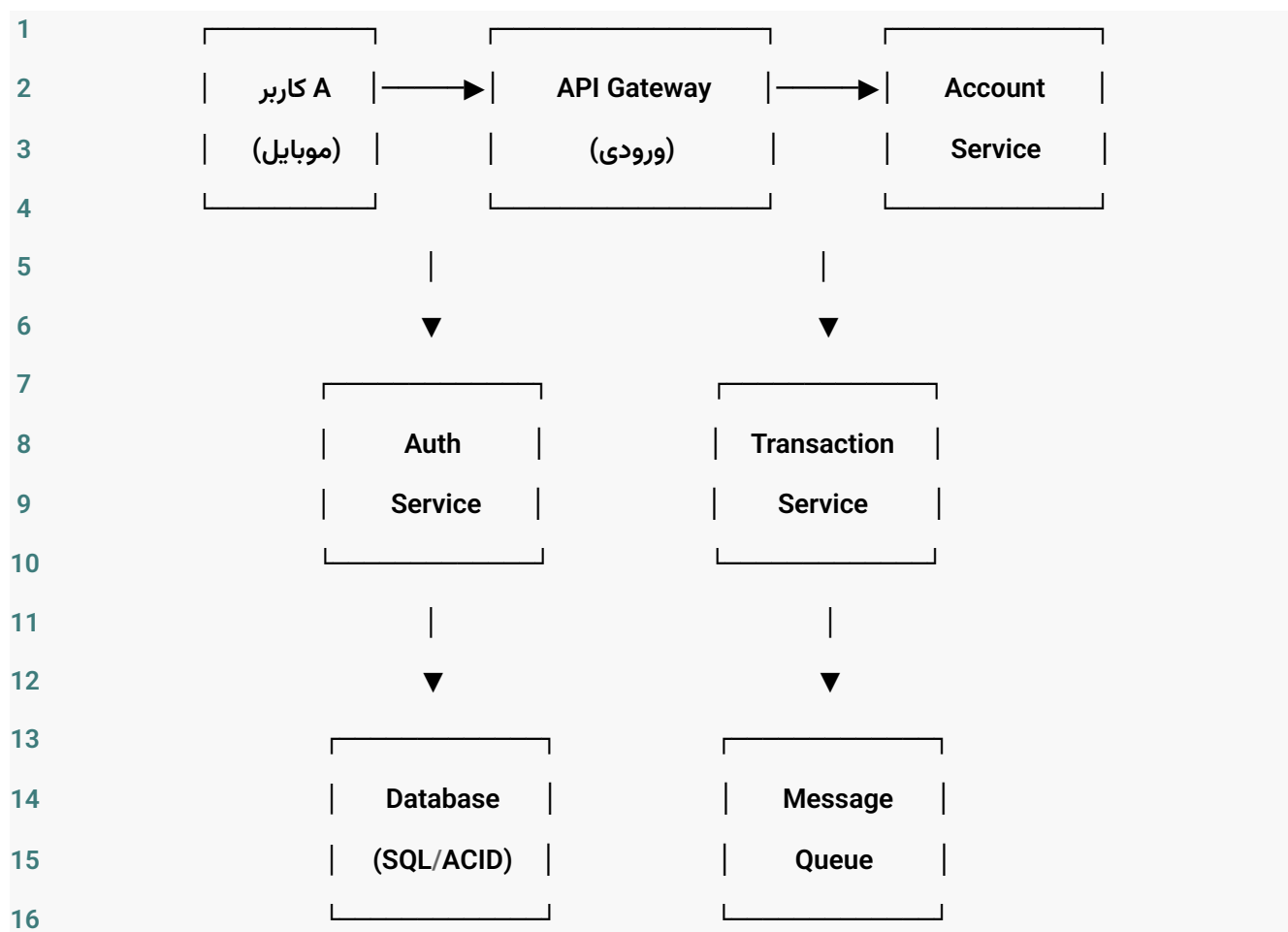
ما می‌خواهیم یک سیستم بانکی طراحی کنیم که کاربران بتوانند:

- حساب بانکی داشته باشند
  - موجودی حساب را ببینند
  - پول انتقال دهند (حواله/کارت به کارت)
  - تاریخچه تراکنش‌ها را مشاهده کنند
- مهم‌ترین چالش: پول نه گم شود، نه دوباره ساخته شود. هر تراکنش باید دقیق باشد.

### تشبیه دنیای واقعی

سیستم بانکی مثل یک گاوصندوق بزرگ با دفتر کل حسابداری است:

- حساب بانکی: یک کشو در گاوصندوق
- تراکنش: یک سند حسابداری که باید مهر و امضا شود
- دفتر کل: دیتابیس که همه تراکنش‌ها را ثبت می‌کند
- کارمند بانک: سرور که درخواست‌ها را پردازش می‌کند
- ممیزی: بازرس که همه اسناد را چک می‌کند



# Account Service و API Gateway

## اجزای اصلی

### ۱. API Gateway (ورودی سیستم)

•وظیفه: اولین نقطه برخورد با کاربر

•تشبیه: درب ورودی بانک با نگهبان

•کارها:

•احراز هویت (Authentication) – چک کردن Token کاربر

•محدودیت نرخ (Rate Limiting) – جلوگیری از حمله

•هدایت درخواست‌ها به سرویس مناسب

### ۲. Account Service (سرویس حساب)

•وظیفه: مدیریت حساب‌های کاربران

•تشبیه: کارمند بانک که حساب‌ها را باز می‌کند

•اطلاعات حساب:

| account_id | user_id | balance    | currency |
|------------|---------|------------|----------|
| 1001       | 501     | 5,000,000  | تومان    |
| 1002       | 502     | 10,000,000 | تومان    |

# Database و Transaction Service

## ۳. Transaction Service (سرویس تراکنش)

•وظیفه: انجام عملیات انتقال وجه

•تشبیه: کارمند باجه که حواله صادر می‌کند

•مراحل:

1. دریافت درخواست انتقال

2. چک کردن موجودی حساب مبدا

3. کم کردن پول از حساب مبدا

4. اضافه کردن پول به حساب مقصد

5. ثبت تراکنش در دیتابیس

## ۴. Database (دیتابیس)

•وظیفه: ذخیره‌سازی حساب‌ها و تراکنش‌ها

•نوع: حتماً SQL (مثل PostgreSQL) به خاطر پشتیبانی از ACID

•جدول تراکنش‌ها:

| txn_id | from_acc | to_acc | amount  | status  |
|--------|----------|--------|---------|---------|
| 90001  | 1001     | 1002   | 500,000 | SUCCESS |
| 90002  | 1002     | 1001   | 200,000 | FAILED  |

# جزئیات حیاتی – جریان انتقال وجه

## جریان انتقال وجه (Core Transaction Flow)

این بخش قلب سیستم است. باید نشان دهید که چطور یک تراکنش را امن و اتمیک می‌کنید.

سناریو: کاربر A می‌خواهد ۵۰۰,۰۰۰ تومان به کاربر B انتقال دهد.

- 1 درخواست انتقال می‌دهد A مرحله ۱: کاربر
- 2 |
- 3 (معتبر است؟ Token) را چک می‌کند A مرحله ۲: سرور هویت کاربر
- 4 |
- 5 چک می‌شود (کافی است؟) A مرحله ۳: موجودی حساب
- 6 |
- 7 در دیتابیس زده می‌شود B و حساب A مرحله ۴: قفل روی ردیف حساب
- 8 |
- 9: مرحله ۵: عملیات اتمیک (در یک تراکنش دیتابیس)
- 10 کم می‌شود (۴,۵۰۰,۰۰۰ → ۵,۰۰۰,۰۰۰) A موجودی -
- 11 زیاد می‌شود (۱۰,۵۰۰,۰۰۰ → ۱۰,۰۰۰,۰۰۰) B موجودی -
- 12 درج می‌شود transactions یک رکورد در جدول -
- 13 |
- 14 (ثبت نهایی) COMMIT → مرحله ۶: اگر همه چیز موفق بود
- 15 |
- 16 مرحله ۷: قفل‌ها آزاد می‌شوند
- 17 |
- 18 به صف پیام فرستاده می‌شود (Event) مرحله ۸: یک رویداد

## مفهوم ACID در بانکداری

🔑 مفهوم ACID (مهم‌ترین مفهوم در سیستم بانکی)

| ویژگی       | توضیح ساده                        | مثال در بانک                                          |
|-------------|-----------------------------------|-------------------------------------------------------|
| Atomicity   | یا همه مراحل انجام شود یا هیچکدام | اگر پول از حساب A کم شد، حتماً به حساب B اضافه شود    |
| Consistency | داده‌ها همیشه معتبر می‌مانند      | موجودی هیچ حسابی منفی نمی‌شود                         |
| Isolation   | تراکنش‌ها از هم جدا هستند         | دو تراکنش همزمان روی یک حساب، همدیگر را خراب نمی‌کنند |
| Durability  | بعد از ثبت، از بین نمی‌رود        | حتی با قطع برق، تراکنش ثبت شده باقی می‌ماند           |

## جلوگیری از تراکنش تکراری (Idempotency)

🛡️ جلوگیری از تراکنش تکراری (Idempotency)

مشکل: اگر کاربر دو بار روی دکمه «انتقال» کلیک کند، چه می‌شود؟

تشبیه: اگر دو بار بگویید «پول را بفرست»، نباید دو بار پول کم شود.

راه حل: هر درخواست انتقال یک idempotency\_key یکتا دارد.

- 1 (UUID مثل) کاربر یک کلید یکتا تولید می‌کند
- 2 |
- 3 درخواست با کلید به سرور می‌رسد
- 4 |
- 5 سرور چک می‌کند: آیا قبلاً با این کلید تراکنشی انجام شده؟
- 6 |
- 7 بله → همان نتیجه قبلی را برمی‌گرداند (پول دوباره کم نمی‌شود) —
- 8 |
- 9 خیر → تراکنش را انجام می‌دهد و کلید را ذخیره می‌کند —

• ابزار: می‌توانیم کلیدها را در Redis با زمان انقضا ذخیره کنیم.

## امنیت در سیستم بانکی

### امنیت (Security)

#### ۱. رمزنگاری (Encryption)

• در حالت استراحت (At Rest): داده‌های حساس مثل شماره حساب و کارت با AES-256 رمزنگاری می‌شوند.

• در حال انتقال (In Transit): همه ارتباطات با TLS 1.3 انجام می‌شود.

• تشبیه: پول را در گاوصندوق قفل‌دار بگذارید و با ماشین ضدگلوله جابجا کنید.

#### ۲. احراز هویت دو مرحله‌ای (2FA)

برای تراکنش‌های بالای یک مبلغ مشخص (مثلاً ۵۰ میلیون تومان)، حتماً کد تایید پیامکی یا اپلیکیشن لازم است.

### ۳. ممیزی (Audit Log)

هر تغییری در سیستم باید ثبت شود: چه کسی، چه زمانی، چه کاری انجام داد.

تشبیه: دوربین مدار بسته در بانک که همه چیز را ضبط می‌کند.

### ۴. تشخیص تقلب (Fraud Detection)

خدمت جداگانه که تراکنش‌ها را تحلیل می‌کند.

الگوهای مشکوک: انتقال مبلغ زیاد در ساعت غیرعادی، به حساب جدید، از IP خارجی.

تشبیه: کارمند امنیتی که رفتار مشتریان را زیر نظر دارد.

## مقیاس‌پذیری و در دسترس بودن

### مقیاس‌پذیری (Scalability)

#### ۱. پارتیشن‌بندی (Sharding)

• مشکل: میلیون‌ها حساب در یک دیتابیس جا نمی‌شود.

• راه حل: تقسیم بر اساس `account_id`

دیتابیس ۱: حساب‌های ۱۰۰۰ تا ۵۰۰۰۰ 1

دیتابیس ۲: حساب‌های ۵۰۰۰۱ تا ۱۰۰۰۰۰ 2

دیتابیس ۳: حساب‌های ۱۰۰۰۰۱ تا ۱۵۰۰۰۰ 3

## ۲. تکثیر (Replication)

• معماری Master-Slave:

• Master: همه نوشتن‌ها اینجا انجام می‌شود.

• Slave: فقط خواندن‌ها (مثل نمایش موجودی) از اینجا انجام می‌شود.

• تشبیه: دفتر اصلی بانک (نوشتن) و شعبه‌های محلی (خواندن).

## ۳. در دسترس بودن بالا (High Availability)

• اگر دیتابیس Master از کار بیفتد، یکی از Slave‌ها سریعاً Master می‌شود.

• تشبیه: اگر مدیر شعبه مرکزی بیمار شود، معاونش سریعاً جای او را می‌گیرد.

# سوالات مصاحبه سیستم بانکی

## ! سوالات مصاحبه‌ای

**سوال ۱: چطور از منفی شدن موجودی جلوگیری می‌کنیم؟**

**جواب:** در تراکنش دیتابیس، قبل از کم کردن پول، شرط  $balance \geq amount$  را چک می‌کنیم. اگر موجودی کافی نبود،

تراکنش Rollback می‌شود.

**سوال ۲:** اگر سرور بعد از کم کردن پول از حساب A اما قبل از اضافه کردن به حساب B از کار بیفتد، چه می‌شود؟

جواب: چون هر دو عملیات در یک تراکنش دیتابیزی ACID انجام می‌شود، اگر سرور بین این دو مرحله از کار بیفتد، دیتابیس به طور خودکار کل تراکنش را Rollback می‌کند. هیچ پولی گم نمی‌شود.

**سوال ۳:** چطور تراکنش‌های بین‌المللی (ارزهای مختلف) را مدیریت می‌کنیم؟

جواب: یک سرویس نرخ تبدیل ارز (Exchange Rate Service) داریم که نرخ لحظه‌ای را می‌گیرد. مبلغ تبدیل می‌شود و بعد تراکنش انجام می‌شود. چالش اصلی: نرخ ارز لحظه‌ای تغییر می‌کند.

**سوال ۴:** تفاوت سیستم بانکی با سیستم فروشگاه‌های چیست؟

جواب:

• در بانک، تراکنش مالی قلب سیستم است.

• سازگاری (Consistency) از در دسترس بودن (Availability) مهم‌تر است.

• در فروشگاه، اگر سبد خرید لحظه‌ای درست نباشد، فاجعه نیست. اما در بانک، اگر موجودی اشتباه باشد، فاجعه است.

**سوال ۵:** چطور با تراکنش‌های همزمان روی یک حساب برخورد می‌کنیم؟

جواب: از قفل ردیف (Row Locking) در دیتابیس استفاده می‌کنیم. وقتی تراکنشی روی حساب A در حال انجام است، تراکنش دیگر باید صبر کند تا قفل آزاد شود.

## خلاصه یک صفحه‌ای



سوال: یک سیستم بانکی طراحی کنید.

پاسخ سطح بالا:

• API Gateway برای ورود و احراز هویت

• Account Service برای مدیریت حساب‌ها

• Transaction Service برای انجام تراکنش‌ها

• دیتابیس (PostgreSQL) SQL با ACID

• Redis برای کش و Idempotency

• Message Queue برای اطلاع‌رسانی غیرهمزمان

• Fraud Detection Service برای تشخیص تقلب

تخمین اعداد:

• ۵۰۰ میلیون کاربر فعال

• هر کاربر ۵ تراکنش در روز → ۲۵۰ میلیون تراکنش در روز

• هر تراکنش متوسط ۱KB فضا → ۲۵۰GB داده در روز!

# سیستم ۷: طراحی سیستم رزرواسیون (مثل Booking.com / علی بابا)

## مشکل چیست؟

ما می‌خواهیم یک سیستم رزرو طراحی کنیم که کاربران بتوانند:

- لیست آیتم‌های قابل رزرو (هتل، بلیط هواپیما، صندلی سینما) را ببینند

- موجودی (اتاق خالی / صندلی خالی) را چک کنند

- یک آیتم را برای خودشان رزرو کنند

- رزرو خود را لغو کنند

مهم‌ترین چالش: دو نفر همزمان آخرین اتاق یا صندلی را رزرو نکنند! (Double Booking)

## تشبیه دنیای واقعی

سیستم رزرو مثل یک سینما با صندلی‌های محدود است:

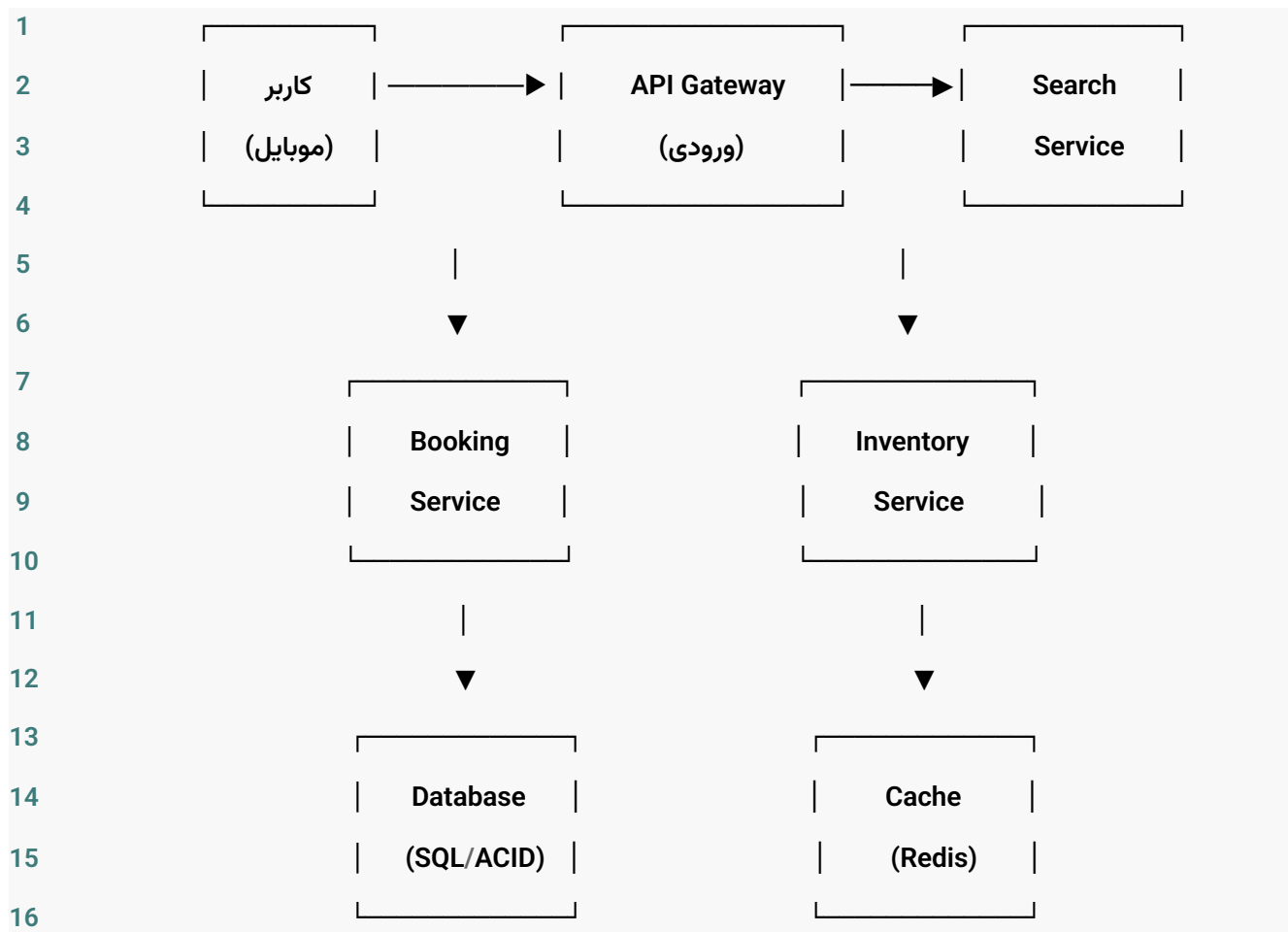
- صندلی سینما: آیتم قابل رزرو (اتاق هتل، صندلی هواپیما)

- بلیط: رزرو شما

- گیشه فروش: سرور که درخواست‌ها را پردازش می‌کند

- نقشه صندلی‌ها: دیتابیس که وضعیت هر صندلی را نشان می‌دهد

- قفل روی صندلی: وقتی کسی صندلی را انتخاب می‌کند، موقتاً قفل می‌شود



## اجزای اصلی

### ۱. Search Service (سرویس جستجو)

•وظیفه: نمایش آیتم‌های موجود بر اساس فیلتر کاربر

•تشبیه: تابلو اعلانات سینما که فیلم‌ها و سانس‌ها را نشان می‌دهد

•کارها:

•دریافت فیلترها (تاریخ، شهر، قیمت، نوع اتاق)

•جستجو در دیتابیس

•نمایش نتایج با وضعیت موجودی

## ۲. Inventory Service (سرویس موجودی)

•وظیفه: مدیریت موجودی آیتم‌ها

•تشبیه: نقشه صندلی‌های سینما که نشان می‌دهد کدام صندلی خالی است

•اطلاعات:

| item_id     | date       | total_count | booked_count |
|-------------|------------|-------------|--------------|
| 101 (هتل A) | 2024-05-01 | 100         | 45           |
| 101 (هتل A) | 2024-05-02 | 100         | 100 ← پر شده |
| 102 (هتل B) | 2024-05-01 | 50          | 10           |

## Database و Booking Service

## ۳. Booking Service (سرویس رزرو)

•وظیفه: ثبت رزرو نهایی

•تشبیه: گیشه فروش بلیط که بلیط را صادر می‌کند

•مراحل:

1. دریافت درخواست رزرو از کاربر

2. چک کردن موجودی با Inventory Service

3. اگر موجود بود، کاهش موجودی

4. ثبت رزرو در دیتابیس

5. صدور بلیط / تاییدیه

## 4. Database (دیتابیس)

• نوع: SQL (مثل MySQL یا PostgreSQL) به خاطر پشتیبانی از تراکنش ACID

• جدول رزروها:

| booking_id | user_id | item_id | date       | status    |
|------------|---------|---------|------------|-----------|
| 70001      | 501     | 101     | 2024-05-01 | CONFIRMED |
| 70002      | 502     | 101     | 2024-05-01 | CANCELLED |
| 70003      | 503     | 102     | 2024-05-02 | PENDING   |

## ⚠ مشکل اصلی: Double Booking (رزرو همزمان)

سناریوی فاجعه: دو کاربر همزمان آخرین اتاق هتل را می‌بینند:

- 1 (از 100 99 booked\_count) اتاق ۱۰۱ را می‌بیند A لحظه ۱: کاربر
- 2 (از 100 99 booked\_count) اتاق ۱۰۱ را می‌بیند B لحظه ۲: کاربر
- 3 لحظه ۳: هر دو رزرو می‌کنند
- 4 ❌ از 101 100 booked\_count = نتیجه

# راه‌حل‌های مدیریت همزمانی در رزرو منابع

## راه‌حل ۱: قفل بدبینانه (Pessimistic Locking)

ایده:

وقتی کاربری در حال رزرو است، ردیف مربوطه را در دیتابیس قفل می‌کنیم. بقیه باید صبر کنند.

تشبیه:

صندلی سینما را با زنجیر موقتاً می‌بندیم تا کسی نتواند بنشیند.

مزیت:

۱۰۰٪ امن

عیب:

کند می‌شود چون همه باید در صف بمانند

```
1 SELECT * FROM inventory
2 WHERE item_id = 101 AND date = '2024-05-01'
3 FOR UPDATE;
4 -- حالا ردیف قفل است. هیچ‌کس دیگر نمی‌تواند آن را تغییر دهد --
```

## راه‌حل ۲: قفل خوش‌بینانه (Optimistic Locking)

ایده:

هر ردیف یک version دارد. موقع تغییر، چک می‌کنیم version عوض نشده باشد.

تشبیه:

روی صندلی یک برچسب زمان می‌چسبانیم. وقتی می‌خواهید بنشینید، چک می‌کنید برچسب همان است که دیدید.

مزیت:

سریع‌تر

عیب:

اگر conflict زیاد باشد، کاربر باید دوباره تلاش کند

1 کاربر A: version = 5 → رزرو می‌کند → version = 6

2 (عوض شده version) رزرو می‌کند → خطا → کاربر B: version = 5

3 باید دوباره تلاش کند B کاربر

## راه حل ۳: Redis با SETNX (پیشنهادی برای سرعت بالا) - پیاده‌سازی در جاوا

ایده:

قبل از رزرو، یک کلید یکتا در Redis می‌سازیم. اگر کلید وجود داشت، یعنی کسی دیگر در حال رزرو است.

تشبیه:

یک چراغ قرمز روی صندلی روشن می‌کنیم. اگر چراغ روشن بود، صندلی اشغال است.

مزیت:

سرعت بسیار بالا به دلیل استفاده از حافظه (In-Memory)

عیب:

• مناسب سناریوهای کوتاه مدت (مثل زمان اتمام خرید)

• نیاز به مدیریت انقضای کلید (TTL) برای جلوگیری از قفل های بی نهایت

1 SET inventory:101:2024-05-01:user501 "locked" NX EX 60

2 -- اگر موفق بود: کاربر قفل را گرفته و می تواند رزرو کند

3 -- اگر ناموفق بود: یکی دیگر در حال رزرو است

# رزرو موقت (Temporary Hold)

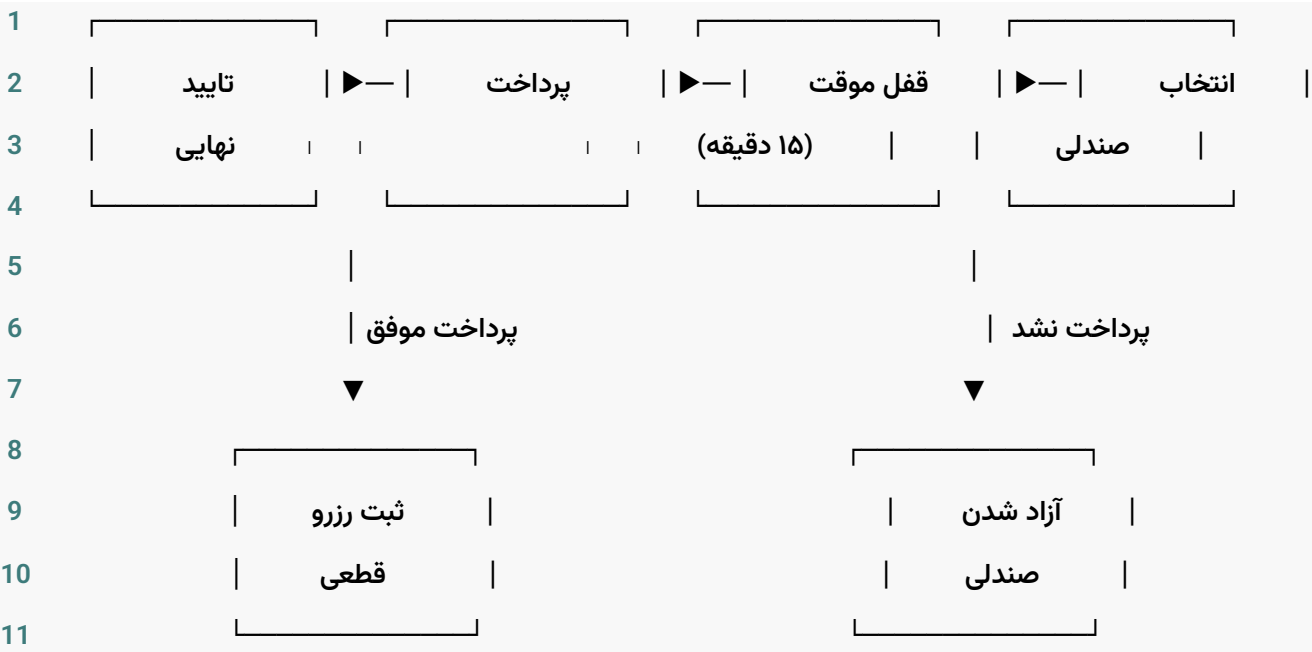
مشکل: کاربر صندلی را انتخاب می کند ولی هنوز پول نداده است. چقدر صبر کنیم؟

راه حل: صندلی را برای مدت کوتاه (مثلاً ۱۵ دقیقه) قفل می کنیم:

• اگر در این ۱۵ دقیقه پول داد → رزرو قطعی می شود

• اگر نداد → قفل آزاد می شود و صندلی دوباره قابل رزرو است

تشبیه: در سینما، صندلی را انتخاب می کنید، به شما ۱۵ دقیقه فرصت می دهند پول بدهید. اگر ندهید، صندلی آزاد می شود.



## مقیاس پذیری

### ۱. کش کردن موجودی (Caching Inventory)

- مشکل: چک کردن موجودی در دیتابیس اصلی کند است.
- راه حل: موجودی را در Redis نگه می داریم.
- تشبیه: یک تابلو دیجیتال که تعداد صندلی های خالی را لحظه ای نشان می دهد.
- نکته: همیشه دیتابیس اصلی مرجع نهایی است. Redis فقط برای سرعت است.

### ۲. شاردینگ بر اساس آیتم (Sharding by Item)

- مشکل: میلیون ها آیتم قابل رزرو داریم.
- راه حل: تقسیم بر اساس item\_id

دیتابیس ۱: آیتم های ۱ تا ۱۰۰۰۰۰

دیتابیس ۲: آیتم های ۱۰۰۰۰۱ تا ۲۰۰۰۰۰

### ۳. جداسازی رزروهای قدیمی

- رزروهای ۶ ماه اخیر: در دیتابیس اصلی (سریع)
- رزروهای قدیمی تر: در دیتابیس بایگانی (ارزان)

## ! سوالات مصاحبه‌ای

**سوال ۱: اگر کاربر رزرو کند ولی پرداخت ناموفق باشد، چه می‌شود؟**

جواب:

- رزرو با وضعیت PENDING ثبت می‌شود
- قفل موقت (مثلاً ۱۵ دقیقه) فعال است
- اگر پرداخت ناموفق بود، قفل آزاد می‌شود
- موجودی به حالت اول برمی‌گردد
- رزرو با وضعیت FAILED ذخیره می‌شود

**سوال ۲: چطور از رزرو بیش از ظرفیت جلوگیری می‌کنیم؟**

جواب:

- از قفل‌گذاری (Locking) استفاده می‌کنیم
- قبل از ثبت، شرط  $\text{booked\_count} < \text{total\_count}$  را چک می‌کنیم
- عملیات کاهش موجودی و ثبت رزرو در یک تراکنش ACID انجام می‌شود

**سوال ۳: اگر دو کاربر همزمان آخرین صندلی را بخواهند چه می‌شود؟**

جواب:

• با قفل بدبینانه (Pessimistic): یکی از آنها منتظر می ماند تا قفل آزاد شود. نفر اول رزرو می کند، نفر دوم خطای «ظرفیت تکمیل» می گیرد.

• با قفل خوش بینانه (Optimistic): نفر دوم باید دوباره تلاش کند و موجودی جدید را ببیند.

## سوال ۴: چطور رزرو گروهی (مثلاً ۵ اتاق با هم) انجام دهیم؟

جواب:

• همه ۵ اتاق باید در یک تراکنش ACID رزرو شوند

• اگر حتی یک اتاق موجود نبود، کل تراکنش Rollback می شود

• به کاربر پیام «رزرو گروهی ناموفق» نمایش داده می شود

## سوال ۵: تفاوت سیستم رزرو با سیستم فروشگاه چیست؟

جواب:

• در رزرو، موجودی محدود و تاریخ محور است (اتاق هتل برای تاریخ خاص)

• در فروشگاه، موجودی فیزیکی است ولی تاریخ محور نیست

• در رزرو، زمان نقش کلیدی دارد (قفل موقت، انقضای رزرو)

## خلاصه یک صفحه ای



سوال: یک سیستم رزرو (هتل/بلیط) طراحی کنید.

پاسخ سطح بالا:

• API Gateway برای ورود

• Search Service برای نمایش آیتم‌های موجود

• Inventory Service برای مدیریت موجودی

• Booking Service برای ثبت رزرو

• دیتابیس SQL با ACID

• Redis برای کش موجودی و قفل موقت

• Message Queue برای اطلاع‌رسانی

تخمین اعداد:

• ۱۰ میلیون آیتم قابل رزرو

• ۵۰۰ هزار رزرو در روز

• هر رزرو متوسط ۵۰۰MB → ۱KB داده در روز

• نرخ تبدیل جستجو به رزرو: حدود ۲٪

---

## سیستم ۸: طراحی سیستم چت زنده / پشتیبانی آنلاین (Live Chat / Customer Support)

### مشکل چیست؟

ما می‌خواهیم یک سیستم چت زنده طراحی کنیم که:

• کاربران بتوانند با پشتیبانی صحبت کنند (مثل چت آنلاین دیجی‌کالا یا اسنپ)

• اپراتورها بتوانند همزمان به چند کاربر پاسخ دهند

• اگر اپراتور آزاد نبود، کاربر در صف انتظار بماند

• تاریخچه چت برای پیگیری بعدی ذخیره شود

تفاوت با پیام‌رسان (واتساپ): در پیام‌رسان، دو کاربر عادی با هم چت می‌کنند. در سیستم پشتیبانی، یک اپراتور (کارمند) به چند کاربر پاسخ می‌دهد و صف انتظار وجود دارد.

## تشبیه دنیای واقعی 🏠

سیستم پشتیبانی آنلاین مثل مرکز تماس (Call Center) است:

• کاربر: مشتری که زنگ زده و منتظر پاسخ است

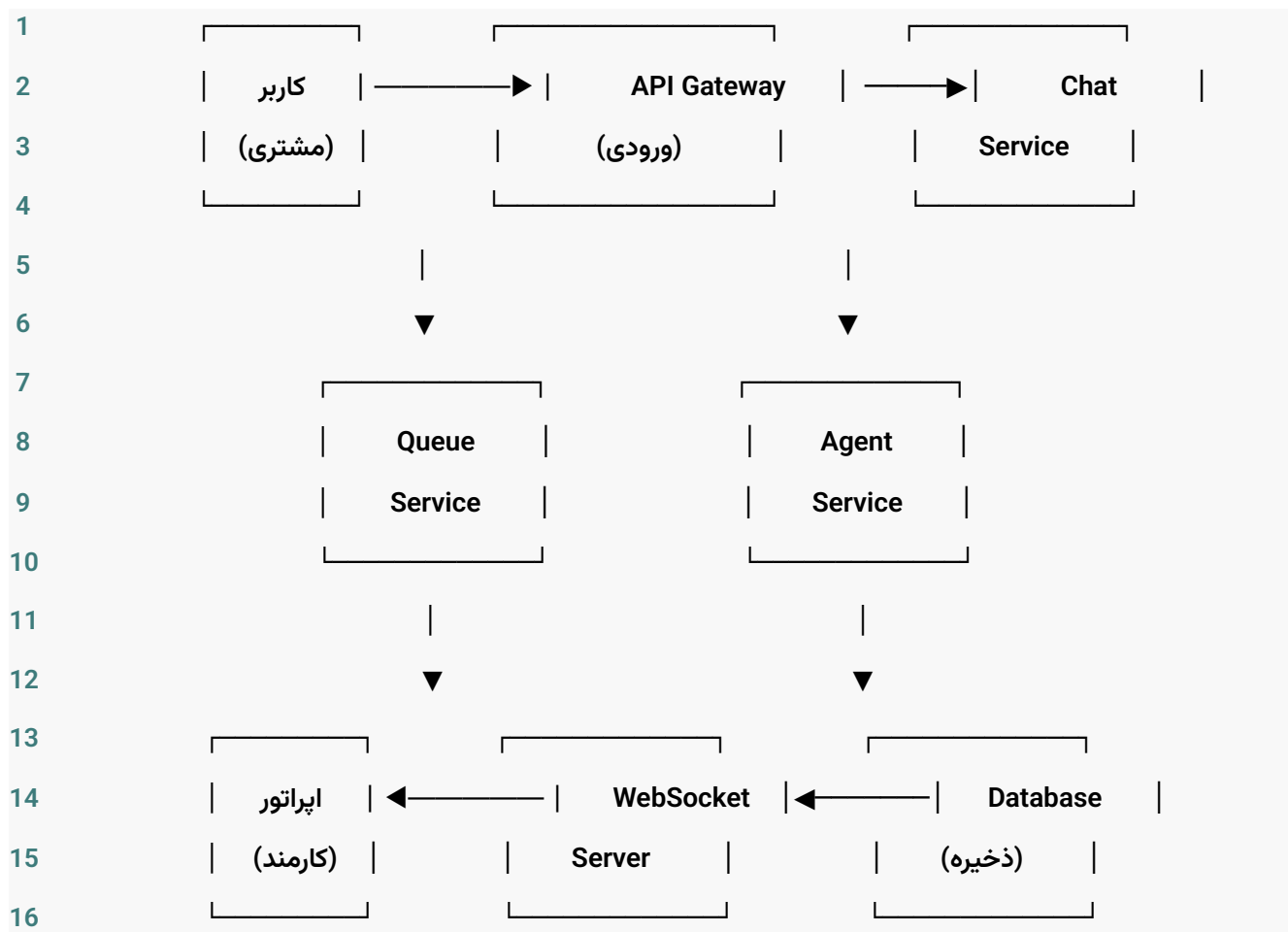
• اپراتور: کارمند پشتیبانی که به مشتریان پاسخ می‌دهد

• صف انتظار: مشتریانی که پشت خط منتظرند

• مدیر شیفت: سیستمی که مشتریان را به اپراتورهای آزاد متصل می‌کند

• ضبط مکالمات: دیتابیس که چت‌ها را ذخیره می‌کند

## معماری کلی 📊



## اجزای اصلی

### ۱. Chat Service (سرویس چت)

•وظیفه: مدیریت پیام‌های چت بین کاربر و اپراتور

•تشبیه: خط تلفن که صدا را منتقل می‌کند

• کارها:

• دریافت پیام از کاربر

• ارسال پیام به اپراتور مربوطه

• ذخیره پیام در دیتابیس

• نمایش وضعیت «در حال تایپ...»

## ۲. Queue Service (سرویس صف انتظار)

• وظیفه: مدیریت صف کاربران منتظر

• تشبیه: صف پشت خط تلفن که به ترتیب وصل می‌شوند

• اطلاعات:

| queue_id | user_id | entered_at | priority |
|----------|---------|------------|----------|
| 1        | 501     | 14:30:00   | normal   |
| 2        | 502     | 14:31:15   | high     |
| 3        | 503     | 14:32:45   | normal   |

## ۳. Agent Service (سرویس اپراتور)

• وظیفه: مدیریت اپراتورها و وضعیت آنها

• تشبیه: مدیر مرکز تماس که می‌داند کدام کارمند آزاد است

• وضعیت اپراتور:

| agent_id | name       | status    | current_chat |
|----------|------------|-----------|--------------|
| 8001     | علی رضایی  | AVAILABLE | null         |
| 8002     | مریم احمدی | BUSY      | 501          |
| 8003     | حسین کریمی | OFFLINE   | null         |

## WebSocket و ارتباط لحظه‌ای

### ۴. WebSocket Server (سرور اتصال لحظه‌ای)

•وظیفه: ارتباط دوطرفه و لحظه‌ای بین کاربر و اپراتور

•تشبیه: تلفن که هر دو طرف می‌توانند همزمان صحبت کنند

•تفاوت با HTTP:

•HTTP: مثل نامه فرستادن (بفرست و منتظر جواب باش)

•WebSocket: مثل تلفن (همیشه وصل هستی، هر لحظه می‌توانی صحبت کنی)

### ۵. Database (دیتابیس)

•نوع: ترکیبی از SQL و Redis

•SQL: برای ذخیره تاریخچه چت‌ها

•Redis: برای ذخیره وضعیت لحظه‌ای (چه کسی آنلاین است، چه کسی در حال چت است)

جدول چت‌ها:

| chat_id | user_id | agent_id | message    | timestamp |
|---------|---------|----------|------------|-----------|
| 50001   | 501     | 8002     | سلام       | 14:35:00  |
| 50001   | 8002    | 501      | سلام چطور؟ | 14:35:10  |
| 50002   | 502     | 8001     | مشکل سفارش | 14:36:00  |

## جریان اتصال کاربر به اپراتور

این قلب سیستم پشتیبانی است. بیاید ببینیم از لحظه ورود کاربر چه اتفاقی می‌افتد:

- 1 مرحله ۱: کاربر وارد صفحه چت پشتیبانی می‌شود
- 2 |
- 3 مرحله ۲: کاربر دکمه "شروع چت" را می‌زند
- 4 |
- 5 قرار می‌دهد Queue Service کاربر را در Chat Service: مرحله ۳
- 6 |
- 7 وجود دارد؟ (AVAILABLE) مرحله ۴: سیستم چک می‌کند آیا اپراتور آزاد
- 8 |
- 9 بله → کاربر به اپراتور آزاد متصل می‌شود
- 10 | تغییر می‌کند BUSY وضعیت اپراتور به -
- 11 | کاربر از صف خارج می‌شود -
- 12 | بین آنها برقرار می‌شود WebSocket -
- 13 |
- 14 خیر → کاربر در صف می‌ماند
- 15 در صف هستید" را می‌بیند X پیام "شما نفر -
- 16 وقتی اپراتور آزاد شد، به او متصل می‌شود -

تشبیه: مثل تماس با مرکز پشتیبانی تلفنی. اول پیام "همه اپراتورها مشغول هستند، لطفاً منتظر بمانید" را می‌شنوید، بعد به محض آزاد شدن یک اپراتور، وصل می‌شوید.

## مدیریت صف (Queue Management)

### ۱. صف عادی (FIFO - First In First Out)

• تشبیه: صف نانوايي - هر کس زودتر آمده، زودتر نوبتش می‌شود

•مزیت: عادلانه است

## ۲. اولویت‌بندی (Priority Queue)

•تشبیه: در بیمارستان، بیمار اورژانسی زودتر از بیمار عادی ویزیت می‌شود

•کاربرد:

•کاربران VIP (خرید زیاد) اولویت بالاتر دارند

•مشکل قطعی پرداخت (که فوری است) نسبت به سوال عمومی اولویت دارد

1 صف اولویت بالا: [کاربر ۱۰۵] [کاربر ۱۰۸] ← اینها زودتر وصل می‌شوند

2 صف عادی: [کاربر ۱۰۱] [کاربر ۱۰۲] [کاربر ۱۰۳]

## ۳. حداکثر زمان انتظار (Timeout)

اگر کاربر ۱۰ دقیقه در صف بماند چه می‌شود؟

•گزینه ۱: پیام "بعداً تماس بگیرید" نمایش داده می‌شود

•گزینه ۲: سیستم یک تیکت (Ticket) می‌سازد و کاربر از صف خارج می‌شود

## قابلیت‌های ویژه ✨

### ۱. وضعیت "در حال تایپ" (Typing Indicator)

•تشبیه: صدای تق تق کیبورد در تلفن که نشان می‌دهد طرف مقابل در حال نوشتن است

•پیاده‌سازی: وقتی کاربر شروع به تایپ می‌کند، یک پیام کوتاه WebSocket می‌فرستد: "کاربر در حال تایپ است"

• اپراتور: یک انیمیشن سه نقطه (...) می بیند

## ۲. انتقال چت (Transfer Chat)

• مشکل: اپراتور نمی تواند مشکل تخصصی (مثلاً مشکل پرداخت) را حل کند

• راه حل: چت را به اپراتور متخصص منتقل می کند

• تشبیه: در مرکز تماس، اگر مشکل فنی است، به واحد فنی منتقل می کنند

## ۳. چت گروهی با چند اپراتور

• مشکل: مشکل پیچیده است و نیاز به نظر چند کارشناس دارد

• راه حل: یک چت گروهی با چند اپراتور و یک کاربر

• تشبیه: جلسه با چند کارشناس برای حل مشکل شما

## ۴. ربات چت (Chatbot)

• کاربرد: پاسخ به سوالات پرتکرار (ساعات کاری، وضعیت سفارش)

• تشبیه: منشی تلفنی که اول پاسخ می دهد و اگر نتوانست حل کند، به اپراتور انسانی منتقل می کند

**امنیت** 

## ۱. احراز هویت

• کاربر: باید لاگین کرده باشد (Token معتبر)

• اپراتور: باید با پنل مدیریت و سطح دسترسی وارد شده باشد

• تشبیه: کارت شناسایی کارمند برای ورود به اتاق پشتیبانی

## ۲. حریم خصوصی

• کاربر A نباید چت کاربر B را ببیند

• اپراتور فقط به چت‌های مربوط به خودش دسترسی دارد

• تشبیه: هر مکالمه تلفنی فقط بین همان دو نفر است

## ۳. ممیزی (Audit Log)

• همه چت‌ها ذخیره می‌شوند

• مدیر می‌تواند چت‌ها را بررسی کند (برای کنترل کیفیت)

• تشبیه: ضبط مکالمات در مرکز تماس برای آموزش و کنترل کیفیت

## مقیاس‌پذیری



## ۱. توزیع اپراتورها

• مشکل: ۱۰۰۰ اپراتور همزمان آنلاین هستند

• راه‌حل: اپراتورها بین چند سرور توزیع می‌شوند

• هر سرور مسئول تعدادی اپراتور است

## ۲. کش وضعیت اپراتورها

- وضعیت اپراتور (آزاد/مشغول/آفلاین) در Redis ذخیره می‌شود
- تشبیه: تابلو دیجیتال که نشان می‌دهد هر کارمند کجاست و چه می‌کند
- مزیت: سریع‌تر از دیتابیس SQL است

## ۳. آرشیو چت‌های قدیمی

- چت‌های ۳ ماه اخیر: در دیتابیس اصلی
- چت‌های قدیمی‌تر: در دیتابیس بایگانی (Cold Storage)

## ! سوالات مصاحبه‌ای

**سوال ۱: چطور یک کاربر را به نزدیک‌ترین اپراتور آزاد متصل می‌کنیم؟**

جواب:

- از Redis برای نگهداری لیست اپراتورهای آزاد استفاده می‌کنیم
- وقتی کاربر وارد صف می‌شود، سیستم اولین اپراتور AVAILABLE را انتخاب می‌کند
- معیار انتخاب: وضعیت (AVAILABLE)، تعداد چت‌های فعال، تخصص

**سوال ۲: اگر اپراتور وسط چت اینترنتش قطع شود، چه می‌شود؟**

جواب:

- سیستم با Heartbeat متوجه قطعی می‌شود
- چت به اپراتور آزاد دیگری منتقل می‌شود
- کاربر پیام "اپراتور جدید به چت پیوست" را می‌بیند
- تاریخچه چت برای اپراتور جدید نمایش داده می‌شود

### سوال ۳: چطور از اسپم و پیام‌های تکراری جلوگیری می‌کنیم؟

جواب:

- Rate Limiting: هر کاربر حداکثر ۱۰ پیام در دقیقه
- فیلتر کلمات نامناسب
- اگر کاربر ۳ بار پیام تکراری فرستاد، اخطار می‌گیرد

### سوال ۴: تفاوت این سیستم با پیام‌رسان (واتساپ) چیست؟

جواب:

| ویژگی         | پیام‌رسان     | چت پشتیبانی            |
|---------------|---------------|------------------------|
| طرفین گفتگو   | دو کاربر عادی | کاربر و اپراتور        |
| صف انتظار     | ندارد         | دارد                   |
| اولویت‌بندی   | ندارد         | دارد (VIP/عادی)        |
| انتقال گفتگو  | ندارد         | دارد (به اپراتور دیگر) |
| مقیاس کاربران | میلیاردها     | میلیون‌ها              |

### سوال ۵: اگر ۱۰۰۰ کاربر همزمان وارد صف شوند، چه می‌شود؟

جواب:

• صف در Redis ذخیره می‌شود (سریع)

• به هر کاربر شماره صف داده می‌شود

• کاربرانی که بیش از ۱۰ دقیقه منتظرند، پیام "بعداً مراجعه کنید" می‌گیرند

• سیستم Auto-scaling اپراتورهای رباتیک را فعال می‌کند

## خلاصه یک صفحه‌ای



سوال: یک سیستم چت زنده / پشتیبانی آنلاین طراحی کنید.

پاسخ سطح بالا:

• API Gateway برای ورود کاربران

• Chat Service برای مدیریت پیام‌ها

• Queue Service برای صف انتظار

• Agent Service برای مدیریت اپراتورها

• WebSocket Server برای ارتباط لحظه‌ای

• Redis برای وضعیت اپراتورها و صف

• دیتابیس SQL برای تاریخچه چت‌ها

تخمین اعداد:

• ۱۰۰۰ میلیون کاربر

• ۱۰,۰۰۰ اپراتور همزمان

• ۱۰ میلیون چت در روز

• هر چت متوسط ۲۰ پیام → ۲۰ میلیون پیام در روز

## جمع‌بندی نهایی هر ۸ سیستم

| # | سیستم         | چالش اصلی                 | دیتابیس اصلی  | ابزار کلیدی      |
|---|---------------|---------------------------|---------------|------------------|
| ۱ | URL Shortener | تولید کد یکتا             | SQL           | Base62, Redis    |
| ۲ | پیام‌رسان     | ارتباط لحظه‌ای            | SQL + NoSQL   | WebSocket, Queue |
| ۳ | شبکه اجتماعی  | تایم‌لاین سریع            | SQL + NoSQL   | Redis, CDN       |
| ۴ | فروشگاهی      | مدیریت موجودی             | SQL (ACID)    | Redis, Queue     |
| ۵ | اشتراک ویدیو  | ذخیره و پخش فایل          | Metadata: SQL | S3, CDN, HLS     |
| ۶ | بانکی         | تراکنش اتمیک              | SQL (ACID)    | Locking, Audit   |
| ۷ | رزرواسیون     | جلوگیری از Double Booking | SQL (ACID)    | Locking, Redis   |
| ۸ | چت پشتیبانی   | صف انتظار و اتصال         | SQL + Redis   | WebSocket, Queue |